2026 IEEE International Symposium on High-Performance Computer Architecture (HPCA)

μShare: Non-Intrusive Kernel Co-Locating on NVIDIA GPUs

Wenhao Huang[1], Zhaolin Duan[1], Laiping Zhao[1*], Yuhao Zhang[1], Yanjie Wang[1], Yiming Li[1], Yihan Wang[1], Yichi Chen[1], Zhihang Tang[1], Kang Chen[2], Deze Zeng[3], Wenxin Li[1], and Keqiu Li[1]
[1]Tianjin University [2]Tsinghua University [3]China University of Geosciences
{hwh, duanzhaol, laiping, yuhaozhang, wangyanjie, l_ym, ehein, chenyichi, tangzhihang, toliwenxin, keqiu}@tju.edu.cn, chenkang@tsinghua.edu.cn, deze@cug.edu.cn

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408561

***Abstract*—The hardware scheduler on NVIDIA GPUs is highly inefficient in utilizing micro-architectural hardware resources. It places blocks from the same kernel within the same GPU Streaming Multiprocessor (SM) core, resulting in a stacking co-locating problem, where identical blocks are placed within the same SM core, saturating only a subset of intra-SM hardware resources while leaving others underutilized.**

The primary challenge in addressing this issue is that the NVIDIA hardware is closed-source, preventing us from directly modifying the hardware scheduler. To bridge the semantic gap between the resource demands of kernels and the scheduler, we introduce *μShare*, which enables intra-SM scattered co-locating of kernels through a non-intrusive *half-plus blocksize shaping* method. It shapes the blocksize of kernels to a half-plus blocksize (i.e., slightly more than half of the SM's thread capacity), scattering identical blocks of the same kernel across different SMs. It further adopts a *time-shifted launching* method to reduce intra-SM resource contention. Compared to state-of-the-art systems, *μShare* does not require intrusive modifications to hardware or kernel code, yet it can still improve inference throughput by 26.90%-54.09% and increases low-level hardware utilization by 38.53%–61.15%.

I. Introduction

Software-defined resource control can leverage the open interfaces provided by hardware to improve resource efficiency [18], [20], [31], [45]. For example, Intel's Resource Director Technology (RDT) [25] supports fine-grained partitioning of shared CPU-socket resources, allowing multiple applications to coexist with minimal interference. Similarly, GPUs offer inter-SM (Streaming Multiprocessor) isolation mechanisms (e.g., MIG [30], MPS [32], and CU masking [1]), and some systems leverage these techniques to co-locate multiple tasks on a single GPU with lower interference [5], [6], [10], [48], [53], [55], [57], [59].

Despite these advancements at the inter-SM level, intra-SM resource utilization remains inefficient. Modern GPUs, such as NVIDIA's Ampere-based A100, include 108 SMs, each comprising 32 FP64 cores, 64 FP32 cores, 64 INT32 cores, 4 Tensor cores, 16 SFU units, and 32 load/store (LDST) units. However, NVIDIA GPUs do not expose interfaces for fine-grained resource allocation within individual SMs, resulting in inefficient management of these on-chip resources. Our experiments demonstrate that kernel execution on GPUs frequently exhibits a resource-utilization pattern we term "1 more, 5 less" wherein one hardware resource is heavily utilized while the other five resources remain significantly underutilized. For example, during the execution of the common matrix multiplication kernel in inference workloads, the utilization of Tensor cores is 88.52%, while the average utilization of the other five hardware resources is only 5.45%.

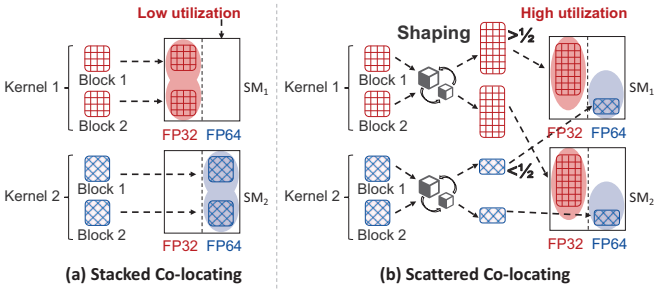

Fig. 1: (a) Stacked co-locating: blocks of a kernel are stacked within the same SM. (b) Scattered co-locating: shaped blocks from different kernels are scattered and co-located within a single SM.

The major reason for low hardware utilization is the *semantic gap between the resource demands of kernels and the resource allocation by the GPU hardware scheduler*, which refers that the actual hardware resource requirements of the kernels are not conveyed to the scheduler, and the scheduler can not perform scheduling based on the hardware demands of multiple kernels. As shown in Figure 1(a), when a kernel is launched, its multiple blocks are queued in the CUDA stream and scheduled one by one in order. This *block-oriented sequential scheduling* often results in the "*stacked co-location*" problem, where identical blocks are placed within the same SM core. As identical blocks share the same resource requirements, this results in low intra-SM hardware utilization.

To improve intra-SM hardware resource utilization, existing works have proposed *intrusive modification* methods. These modifications fall into two main categories: (1) *intrusive modifications to CUDA kernels* [4], [14], [29], [36], [61]–[63], which fuse multiple kernels with complementary resource requirements into a single, more complex kernel, and (2) *intrusive modifications to hardware* [11], [38], [43], where the GPU is redesigned to open up kernel scheduling interfaces, typically validated through simulators. However, in public cloud where NVIDIA GPUs are widely used, and with the

*Corresponding Author

hardware scheduler being **closed-source**, intrusive hardware modifications and cross-user kernel fusion are not feasible.

Thus, the first challenge in bridging the semantic gap is determining how to **non-intrusively** convey kernel resource demands and enable the GPU hardware scheduler to perform resource-coupled scheduling effectively. Under the closed-source nature of NVIDIA GPUs, the only thing we can do is to perform control-based optimizations before the kernels are launched on the GPU, such as configuring the launching timing and kernel parameters (e.g., blocksize, which refers to the thread count for each block, and shared memory). Thus, the challenge is divided into three subproblems: (1) The closed-source nature of NVIDIA GPUs prevents us from fully understanding the hardware scheduler's strategy; how can modifying kernel parameters enable the hardware scheduler to achieve intra-SM co-location? (2) CUDA kernels have multiple configurable parameters; which kernel parameters can serve as a general-purpose solution to be modified? (3) Certain CUDA kernels (e.g., cuDNN) are closed-source and unmodifiable; how should kernels with unmodifiable parameters be co-located?

To address this challenge, we propose a hardware–software co-design approach that enables intra-SM co-location of kernels without modifying kernel code or GPU hardware. This method consists of the following key components: For the first subproblem, we perform extensive profiling under diverse parameter configurations and execution scenarios (i.e., exclusive and concurrent), collecting data on block placements within SMs and their execution timings, which enables us to further understand the block scheduling process. Specifically, under the GPU's left-over scheduling strategy [36], [51], [52] (i.e., blocks are scheduled to SM cores that meet the required thread count), we propose the "*half-plus blocksize shaping*" technique to separate blocks of kernels with similar dominant resource demands. As shown in Figure 1(b), by shaping the kernel's *blocksize* to *half-plus* (i.e., slightly more than half of the SM's thread capacity), multiple blocks from the same kernel are scattered across different SMs, the remaining threads in each SM can be allocated to blocks of other kernels, thereby improving intra-SM hardware utilization. For the second subproblem, our profiling reveals that the blocksize parameter is the only general-purpose solution that effectively scatters identical blocks across different SMs. In contrast, other parameters, such as shared memory, do not achieve this effect. For the third subproblem, we propose the "*time-shifted launching*" technique to co-launch kernels with different resource requirements. For kernels with unmodifiable parameters, we control their launch timing to enable scattered co-location with other kernels that have modifiable parameters.

Based on the "*scattered co-locating*" method, we develop *µShare*, an inference system that bridges the semantic gap between kernel resource demands and hardware scheduler resource allocation. *µShare* is the first to simultaneously achieve three capabilities: kernel-level control, intra-SM co-location, and non-intrusiveness to hardware and kernels' code. *µShare* operates in Linux user space, without generating any additional CPU-GPU communication, and without requiring the reading or modification of any user code, it can increase the system throughput of an NVIDIA GPU from 1722 QPS (queries per second) to 3046 QPS, and improve the aggregate utilization of the six intra-SM hardware units from 56.22% to 90.60%, with only several nanoseconds of control overhead per kernel.

Contributions. Our contributions are as follows:

- **Analysis and discovery**: The primary cause of low resource utilization of GPU SMs is the "*stacked co-locating*" problem in the hardware scheduler.
- **Half-plus blocksize shaping method**: Achieves kernel-level scattered co-location by simply shaping the blocksize of some kernels to half-plus of SM thread capacity.
- ***µShare* prototype**: A non-invasive system including kernel interception, parameter modification, scattered co-locating and inference latency guarantee.
- **Experimental evaluations**: Compared to state-of-the-art, *µShare* improves throughput by 26.90%-54.09% and increases low-level hardware utilization by 38.53%–61.15%, while guaranteeing the latency SLO (Service Level Objective).

II. Background & Motivation

A. The GPU Scheduling Process

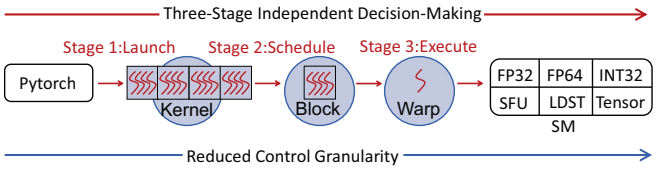

Fig. 2: A kernel's lifecycle includes three distinct stages: launching, scheduling, and execution.

The execution process of a CUDA kernel can be divided into three stages: (1) *launching*, (2) *scheduling*, and (3) *execution*. As shown in Figure 2, the three stages proceed sequentially, each implementing a specific task: the launching of *kernels*, the scheduling of *blocks*, and the execution of *warps*.

In the *launching* stage, a *kernel* is launched by the inference framework (e.g., TF-Serving [49], Pytorch [40], TVM [7] and TensorRT [50]) on the CPU side, and it reaches the CUDA stream via the GPU's Command Processor. A *kernel* is a specialized function executed on the GPU, serving as a basic unit of the inference model. For example, *matrix multiplication* is one of the most commonly used kernels in inference models.

In the *scheduling* stage, each *kernel* is divided into several equally sized *blocks*, with each *block*'s size referred to as the *blocksize*. The GPU's dispatch unit, implemented in hardware [3], will assign blocks to SM cores based on the availability of resources.

In the *execution* stage, each block is further divided into several equally sized *warps*, i.e., a set of 32 threads executed together through single instruction multiple threads. A *warp* is the basic unit of execution. The GPU's *warp* scheduler, also implemented in hardware [3], will select warps from blocks and start the execution.

As the three decision-making stages throughout the execution process are distinct and the granularity of the scheduled objects gradually decreases, it is highly challenging to achieve the optimal resource-efficient scheduling [44].

B. Low Utilization of Low-Level Hardware

To understand kernel's use of low-level hardware, we select 10 commonly used models from MLPerf [42] and Pytorch benchmark [39], as shown in Table III. The maximum batch size for each model is determined under the condition that inference latency does not exceed 200 milliseconds (i.e., the common SLO for inference services in production environments [55]). All data are collected on NVIDIA A40 GPU.

Observation #1: *During the scheduling of stage #2, multiple blocks from the same kernel are stacked and co-located in SM cores. The next kernel's blocks cannot execute until the current kernel nears completion.*

To evaluate the scheduling of kernels, we select and launch two commonly used kernels in inference tasks: the *vectorized* kernel (i.e., performs vectorized layer normalization, mainly using LDST hardware) and the *roll* kernel (i.e., loops through the elements in the displacement array, mainly using INT32 hardware). These two kernels account for 28.90% of the total invocations among all open-source kernels, and we obtain the SM location and start time for each block by using CUDA inline assembly instructions [24] to read the GPU SM_ID register and the GPU clock counter register, respectively. Figure 3(a) shows the scheduling results. We see that, although both *vectorized* kernel and *roll* kernel are launched concurrently on two CUDA streams, blocks of *vectorized* kernels are scheduled for execution first, leaving *roll* kernel's blocks to wait until *vectorized* kernel's blocks are completed.

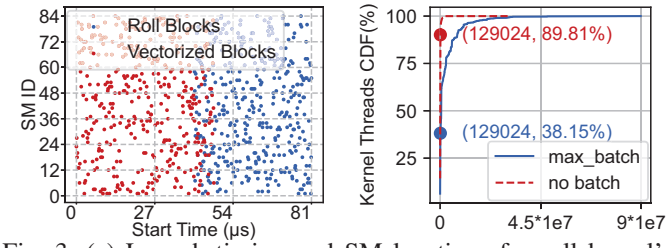

Fig. 3: (a) Launch timing and SM locations for roll kernel's blocks and vectorized kernel's blocks, with roll kernel's blocks executing after nearly all vectorized kernel's blocks have completed. (b) Under max_batch, only 38.15% of kernels have thread requirements less than one GPU, while the remaining kernels fully occupy the GPU.

The reason for the above phenomenon is that after a kernel launch, the GPU's dispatch unit schedules all blocks of a kernel to SM cores [22]. When the number of threads in a kernel exceeds the total thread capacity of the GPU (i.e., the NVIDIA A40 GPU has 129,024 threads), the blocks of a kernel occupy all SM cores. Figure 3(b) shows the statistical analysis of thread counts from 6,802 executions of kernels across the 10 models. Under max_batch, 61.85% have more threads than the GPU can accommodate, and these kernels account for 70.83% of the total execution time. As a result, all blocks of a kernel are stacked and co-located within the SM cores, and the blocks of another kernel cannot begin execution until the current blocks are close to completion.

Observation #2: *During the execution of stage #3, the utilization of six hardware resources by the kernels follows a "1 more, 5 less" pattern. Therefore, under the stacked scheduling of blocks, the overall hardware resource utilization within the SM cores becomes inefficient.*

GPU resource utilization can be measured using two main methods (Figure 4(a)): (1) measure the actual usage time of GPU resources relative to the total time using the NVIDIA-SMI tool [34]; (2) measure the low-level hardware utilization of kernels during the execution using the Nsight Compute tool [33]. NVIDIA-SMI uses the active time ratio as the criterion for GPU utilization, even if only one thread in one SM is active, it will display 100% GPU utilization, significantly exaggerating the actual utilization. We analyze kernel-level resource utilization based on 6,802 executions using the above two tools: NVIDIA-SMI reports 81.16% utilization, while Nsight Compute reports only 9.28% for low-level hardware.

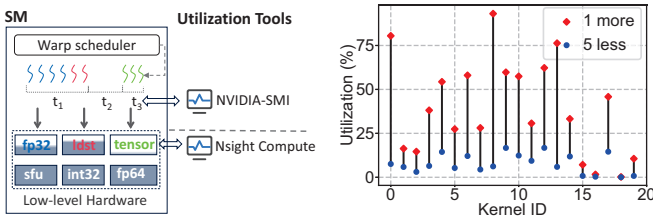

Fig. 4: (a) The schematic diagram of NVIDIA-SMI and Nsight Compute tools. (b) The utilization of the six hardware resources follows a "1 more, 5 less" pattern, where the "1 more" refers to the highest utilization among the six hardware types, and the "5 less" represents the average utilization of the remaining five.

The utilization of six hardware resources by kernels follows a "1 more, 5 less" pattern. Figure 4(b) presents the hardware utilization of the top 20 most frequently executed kernels, totaling 6,063 executions. In the figure, red dots indicate the utilization of the primary hardware resource for each kernel, with an average utilization of 30.19%, while blue dots represent the average utilization of the remaining five hardware types, which is only 5.07%. As all blocks of a kernel have identical hardware resource demands, when a kernel runs on a dedicated SM under stacked co-location, five types of hardware resources typically exhibit low utilization.

C. Blocksize Shaping Improves Utilization

Observation #3: *Blocksize shaping helps: Given that the NVIDIA GPU hardware scheduler is closed-source, shaping the blocksize is the only thing we can do to influence the co-locating decisions. By setting the blocksize to half-plus of SM threads, it becomes indirectly possible for blocks to break the "stacked co-location" pattern.*

As shown in Figure 2, the launching stage of kernels is controlled by the inference framework, where the blocksize parameter is exposed. However, the scheduling and execution stages are managed by closed-source GPU hardware scheduler. Therefore, we can only manipulate the launching stage to indirectly optimize scheduling and execution.

When the remaining available thread capacity inside an SM is greater than the blocksize, the scheduler can schedule the block to the SM. To prevent multiple blocks of the same kernel from being co-located on the same SM core, we can set the blocksize to slightly more than half of the SM's thread capacity (i.e., *half-plus*). This ensures that the combined thread count of any two blocks exceeds the maximum thread limit per SM core, thereby preventing multiple blocks of the same kernel from being scheduled to the same SM.

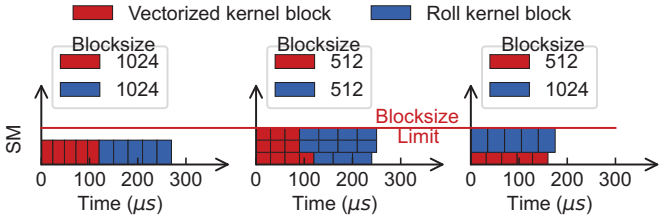

Fig. 5: (a) When both kernels use large blocks: serial execution. (b) When both kernels use small blocks: serial execution. (c) When one kernel uses a large block and the other a small block: parallel execution.

We analyze the execution process within an SM using the *roll* and *vectorized* kernels with different blocksize. As shown in Figure 5(a), when the blocksize of the two kernels is both set to exceed half of the thread capacity (i.e., $blocksize = 1024 > 1536/2$, where the thread limit of an SM in NVIDIA A40 is 1536), after the threads of SM are allocated to the first block, the remaining threads cannot accommodate the second block, the blocks of the two kernels can only execute serially. In Figure 5(b), when the blocksize of the two kernels is set to less than half of the thread capacity (i.e., $blocksize = 512 < 1536/2$), blocks of *vectorized* are stacked and executed first; upon completion, the execution of *roll* begins. However, in Figure 5(c), if one kernel is configured with a large blocksize (i.e., 1024) and the other with a small blocksize (i.e., 512), after placing the large block into an SM, only the small block can fit with the remaining threads, allowing blocks from different kernels to be co-located within the SM. This enables the SM to utilize multiple types of hardware resources simultaneously.

In addition, CUDA allows configuring the amount of additional shared memory during kernel launch; however, our experiments show that large shared memory does not enable co-location of different kernels within the same SM. Hence, we focus solely on modifying the blocksize.

Observation #4: *Before a kernel is launched in stage #1, its blocksize is typically preset to a non-half-plus value. Since the available resources within SM cores fluctuate dynamically, this static setting can lead to inefficient use of resources.*

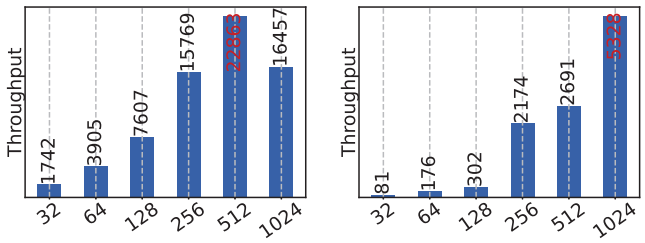

Fig. 6: (a) The blocksize of the roll kernel is statically preset to 512, which results in the highest resource efficiency for this kernel. (b)When the roll kernel is executed concurrently with a preceding vectorized kernel, the blocksize with the highest resource efficiency for the roll kernel changes to 1024.

Existing inference frameworks use a statically preset method to set the blocksize of kernels. By default, typical frameworks like PyTorch [40], TVM [7] and TensorRT [50] use an "enumerate and choosing the best" policy to set the blocksize, i.e., evaluating the resource efficiency at various blocksize and choose the one with the highest efficiency. As a result, the blocksize is never set to a *half-plus* value, as it is not divisible by the SM thread count and results in wasted threads during single-kernel execution. For example, as shown in Figure 6(a), PyTorch presets the blocksize of the roll kernel to 512, while the SM thread count is 1,536, achieving the highest throughput of 22,863. This is at least 1.39 times more efficient compared to other blocksize.

However, the available resources within SM cores dynamically change as kernels are continuously scheduled and executed. The statically preset blocksize, initially designed for dedicated GPU usage, no longer achieves optimal throughput. For example, as shown in Figure 6(b), when the *roll* kernel is co-located with a *vectorized* kernel configured with blocksize 256, the blocksize of the *roll* kernel that achieves the highest throughput shifts from 512 to 1,024, yielding at least a 1.98× improvement compared to other blocksize settings. Moreover, in production systems, the static setting of blocksize is susceptible to resource availability, which can prevent the system from achieving higher throughput.

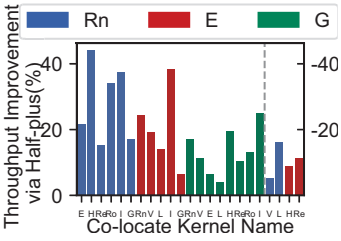

Fig. 7: Throughput improvement via half-plus.

We select three kernels with different dominant hardware to co-locate with other kernels. When the dominant resources differ, the half-plus configuration improves throughput by 19.94% (first 19 bars); otherwise, throughput decreases by 10.37%(last 4 bars).

D. Implications

In summary, we find that the low utilization of low-level hardware within SM cores is mainly due to two reasons: (1) the *stacked co-location* of blocks from the same kernel (**Observation 1**) and (2) the *uneven utilization* of resources

by blocks (**Observation 2**). Existing works achieve intra-SM co-location through intrusive modifications to GPU hardware or kernel code, which are not supported in public cloud environments.

To enable non-intrusive co-location of different kernels, we find that setting the blocksize to *half-plus* of SM threads is a possible way (**Observation 3**). However, existing inference systems typically *statically preset* the blocksize to a non-*half-plus* value, which fails to improve resource efficiency under co-location scenarios (**Observation 4**). Therefore, we design a method to dynamically adjust the blocksize of kernels to enable co-location of different kernels at runtime.

III. Design

In this section, we present the design of *μShare*.

A. Overview

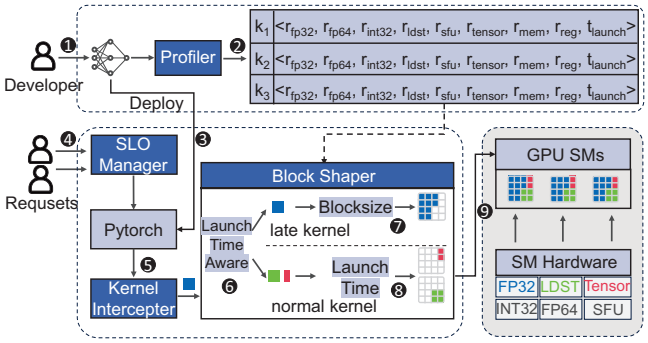

Fig. 8: The system architecture of *μShare*.

The insight of *μShare* is to adjust the blocksize of kernels to "half-plus" of SM thread capacity to achieve scattered co-location of kernels. Figure 8 shows the overall architecture of *μShare*. After an AI model is developed❶, the *profiler* will analyze the model to find the maximum batchsize that meets the SLO (Service Level Objective). Then, under the maximum batchsize, it profiles each kernel's low-level hardware usage as well as its shared memory and register consumption to support the co-location of kernels with complementary resource demands❷.

After profiling, the model is deployed to the inference framework (e.g., Pytorch)❸. When user requests arrive❹, the *batch manager* batches multiple requests and sends the batch to PyTorch for execution. PyTorch then sequentially launches the kernels to the GPU for computation. To avoid intrusive modifications to Pytorch and CUDA, we design a *kernel interceptor* which can intercept the launched kernel❺ and send it to *shaper*❻. For kernels with a late launch time that could potentially cause SLO violations, the *shaper* modifies their blocksize parameters to *half-plus* to accelerate computation❼. For other normal kernels, the *shaper* does not modify their blocksize but adjusts their launch time❽ for a *time-shifted launch*. That is, according to kernel profiles, only kernels that are complementary in resource utilization to those currently executing on the GPU will be launched, while others will be queued. In this way, the blocks with half-plus blocksize can be co-located with those with smaller blocksize for deployment❾ (Observation #3).

B. Kernel Profiler

The *profiler* mainly characterizes the resource demands of each kernel to facilitate the co-locating of kernels with complementary resource demands. To meet the latency SLO, it also profiles the launch time of each kernel when executed with the maximum batchsize allowed within the SLO. In particular, it runs the inference with the maximum batchsize in model exclusive GPU environment and records a 9-tuple for each kernel k:

$$k = \{r^k_{fp32}, r^k_{fp64}, r^k_{int32}, r^k_{ldst}, r^k_{sfu}, r^k_{tensor}, r^k_{mem}, r^k_{reg}, t^k_{launch}\} \quad (1)$$

where $\{r^k_{fp32}, r^k_{fp64}, r^k_{int32}, r^k_{ldst}, r^k_{sfu}, r^k_{tensor}\}$ denotes the low-level hardware utilization during the execution of kernel k, including FP32 core, FP64 core, INT32 core, LD/ST unit, SFU unit, and Tensor core, $\{r^k_{mem}, r^k_{reg}\}$ denotes the shared memory and register usage of kernel k, respectively; and t^k_{launch} denotes its launch time.

We utilize NVIDIA's Night Compute tool [33] to record the utilization rate of six low-level hardware during the execution of each kernel. Additionally, we utilize NVIDIA's Night Systems tool [35] to record the kernel launch time.

C. Kernel Interceptor

When the inference requests begin execution, their kernels are launched by the inference framework. Then, *μShare* intercepts the launched kernels through its *kernel interceptor* without modifying the kernel code or GPU hardware scheduler. Hence, such a *non-intrusive* design can significantly reduce development complexity.

Since CUDA's kernel launch functions (e.g., cudaLaunchKernel or cublasSgemm in cuBLAS) explicitly expose input parameters, and the addresses of their compiled dynamic link libraries (e.g., libcudart.so, libcublas.so, libcudnn.so) can be obtained easily, it is possible to interception kernels in a non-intrusive way. Specifically, the *kernel intercepter* captures dynamic link libraries using the LD_PRELOAD [27] and the dlopen and dlsym [13] functionalities provided by Unix systems. It creates functions with the same names as CUDA's kernel launch functions. By using LD_PRELOAD, these homonymous functions are loaded first. Then, dlopen and dlsym get the original addresses and input parameters (e.g., blocksize) of the CUDA's kernel launch functions. This allows us to modify the input parameters before passing them back to the original functions, restoring their execution.

Furthermore, CUDA's kernel launch functions can be divided into two categories based on whether the kernel blocksize parameters can be modified. The first category is modifiable, such as the cudaLaunchKernel shown in Listing 1. All CUDA kernels launched using CUDA syntactic sugar <<<gridsize, blocksize, sharedMem, stream>>> are directed to this function. Among these, the blocksize and gridsize

parameters are used for subsequent parameter modification operations, representing the number of threads within a modified block and the number of blocks, respectively. The parameters of this syntactic sugar correspond to the gridDim and blockDim in List 1.

```
extern __host__ cudaError_t CUDARTAPI
    cudaLaunchKernel(
   const void *func,
   dim3 gridDim,
   dim3 blockDim,
   void **args,
   size_t sharedMem,
   cudaStream_t stream
);
```

Listing 1: Blocksize modifiable launch function.

The second category is unmodifiable, which utilize CUDA wrapper libraries (such as cuDNN or cuBLAS) for launching, like the cublasSgemm shown in Listing 2. These functions hide the blocksize parameters within closed-source code. Additionally, kernels that produce incorrect results after blocksize modification are also *unmodifiable* (e.g., tiling-based kernels like Conv2d, which trigger a CUDA internal error when modified).

```
void CUBLASWINAPI cublasSgemm(
   char transa,
   char transb,
   void **args
);
```

Listing 2: Blocksize unmodifiable launch function.

We analyze 67 kernels of 10 models (Table III) during inference, which are executed a total of 6,802 times. Among these, modifiable kernels are executed 3,512 times, accounting for 51.63% of the total executions, while unmodifiable kernels are executed 3,290 times, accounting for 48.37%. Table I and II provide a detailed breakdown of each kernel's name, execution count, and percentage. Since modifiable kernels account for more than half of the total executions, there is significant potential for optimizing kernel co-location.

TABLE I: Statistics of blocksize modifiable kernels

Name	FP32	FP64	INT32	LDST	SFU	Tensor	Count	Time(µs)
RNN Cell	9.39	0.00	12.64	16.05	6.96	0.00	1002	40378
Vec Element	7.91	0.00	9.29	13.06	4.08	0.00	971	159150
Elemwise	11.12	0.00	38.12	17.18	0.00	0.00	947	116440
Layer Norm	13.43	0.00	33.08	58.02	11.03	0.00	128	27497
Histo	1.60	0.00	4.36	2.39	0.00	0.00	80	138
Reduction	23.50	0.00	57.43	32.29	0.00	0.00	66	9974
Roll	20.82	0.00	33.25	12.47	24.94	0.00	44	14713
Indexed Elem	1.21	0.00	0.92	0.29	0.00	0.00	26	141
Other	–	–	–	–	–	–	248	2278
Total	–	–	–	–	–	–	**3512**	**370709**

D. Shaper

While it is not possible to develop a new co-locating policy under closed-source GPU hardware scheduler (**Observation 3**), the *shaper* can indirectly influence the scheduling results by adjusting their *blocksize* and the *relaunch time* (i.e., the time when the *shaper* relaunches the kernel to GPU after it has been intercepted.).

TABLE II: Statistics of blocksize unmodifiable kernels

Name	FP32	FP64	INT32	LDST	SFU	Tensor	Count	Time(µs)
CUTLASS Gemm	5.93	0.00	11.44	17.51	0.00	80.49	1293	223538
CUDNN NCHW	21.36	0.00	54.34	48.78	0.00	0.00	560	55438
BatchNorm	29.79	0.00	5.81	7.64	5.39	0.00	496	44829
CUBLAS Gemv	10.37	0.00	19.29	59.31	0.00	0.00	495	49980
MatMul	2.54	0.26	6.51	16.59	0.00	92.39	141	33665
CUDNN NHWC	7.59	0.00	14.53	55.10	0.00	0.00	116	9560
CUTLASS Comb	1.39	0.00	10.96	18.63	0.00	77.11	99	104
Conv	21.39	0.00	57.75	42.98	8.15	0.00	86	93
Other	–	–	–	–	–	–	4	107
Total	–	–	–	–	–	–	**3290**	**417314**

Denote by O the set of kernels intercepted by the *kernel interceptor*. We divide the set O into two subsets X and Y, such that $O = X \cup Y$ and $X \cap Y = \emptyset$, where X refers to the set of kernels for which we are going to modify their *blocksize*, and Y refers to the remaining set of kernels for which we are going to modify their *relaunch time*. To obtain the set X, we calculate the kernel launch slack:

$$s^k = t^k_{launch} - t^k_{intercept}, \quad \forall k \in O \tag{2}$$

where $t^k_{intercept}$ is the real-time launch time of kernel k, and t^k_{launch} is the profiled launch time of kernel k by the *profiler* (Formula 1). We then sort these kernels in an ascending order of s^k. The first x kernels in this sorted list are added to the set X (i.e., $|X| = x$), while the remaining kernels are added to the set Y. Hence, kernels in set X have tighter latency constraints and we need to reshape their blocksize and launch them immediately.

Half-plus Blocksize Shaping: For a kernel $k_i \in X$, $\forall i \in [1, .., x]$, the *shaper* sets k_i's *blocksize* to *half-plus*: $t_{half} + \alpha$, where t_{half} denotes the half number of the SM thread capacity (e.g., $t_{half} = 768$ in NVIDIA A40) and α is a small positive integer. In this way, it is not possible to place more than one k_i's block in a single SM simultaneously. This enforces the distribution of k_i's blocks across different SMs, avoiding the "*stacked co-location*" problem. Meanwhile, larger *blocksize* improves kernel execution efficiency, reducing SLO violations. Note that, the value of x is determined by the smallest x such that the number of blocks of the first x kernels exceeds the number of SMs.

We consider two principles that guide our definition of α:

(1) The parameter α should be able to reduce resource fragmentation. Since the default *blocksize* is typically powers of 32, such as 32, 64, 128, 256, 512, or 1024, the value of α should also follow this pattern.

(2) The parameter α should be able to reduce SLO violations. When the kernel launch time slack (i.e., s^k in Formula 2) is positive, we set $t_{half} + \alpha$ as the smallest number greater than half of SM thread capacity. For example, since the SM thread capacity is 1,536 in NVIDIA A40, we have $t_{half} + \alpha = 768 + 32 = 800$ (note that 32 is the number of

threads in a warp, which is the smallest unit of execution). When the kernel launch slack s^k is negative, we gradually increase α by 32 to speed up the execution of the kernel.

Time-shifted Launch: For a kernel $k_j \in Y$, $j \in [x+1, .., x+y]$, the *block shaper* sets k_j's *relaunch* time and uses their default *blocksize*. The reason is as follows:

(1) Default *blocksize*: The *blocksize* of all kernels of models (Table III) ranges from 32 to 512, all of which are less than half of the SM thread capacity (Figure 9), making them easily co-locatable with the *half-plus* kernels in set X within the same SM core. Therefore, the *blocksize* of $k_j \in Y$ does not need to be modified.

(2) *Relaunch time*: Kernel k_j is relaunched directly if both of the following conditions hold: For each of the six hardware resource types, the combined utilization of k_i and k_j does not exceed 100%, and available shared memory, registers (i.e., the limiting factors for GPU scheduling other than blocksize, profiled in Formula 1) are sufficient.

Otherwise, k_j waits for β microseconds for a *time-shifted launch* before rechecking the above conditions, and its kernel launch slack is updated according to Formula 2. After updating the slack values and reordering all kernels in the set O based on the updated slack, if k_j moves into the top-x positions of the list, its *blocksize* will be set to *half-plus*, like the other kernels in X, and it will be immediately launched for execution. This process is repeated until all kernels have been launched.

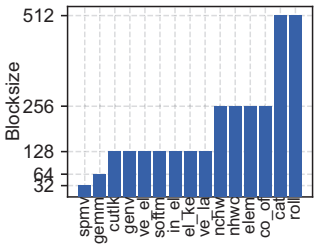

Fig. 9: The default blocksize of kernels is less than half of SM threads.

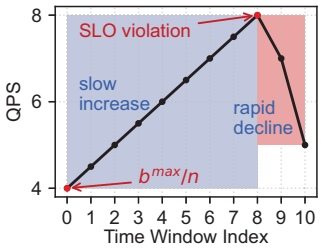

Fig. 10: The control process of batch size after each time window.

E. Batch Manager

The *batch manager* is responsible for managing the *batch size*, which determines how many user requests the system can process in a single inference [15]. Aggregating user requests into a large batch can improve system throughput. However, when the request arrival rate is low, the waiting time required to accumulate a large batch may lead to SLO (Service Level Objective) violations. In addition, when multiple models share a GPU, resource contention between models may increase latency, further risking SLO violations [8], [9]. Therefore, it is necessary to adjust the batch size in real time based on both interference and request frequency.

We adopt a *feedback-based* strategy for adjusting the *batch-size*, i.e., adjusting the *batchsize* based on the monitored real-time latency. If the monitored latency is less than the SLO, increase the *batchsize*. Otherwise, decrease the *batchsize*. To avoid oscillations caused by short-term workload bursts, we monitor the response latency over a time window and employ an *exponential decay* algorithm [12] to derive the SLO slack (denoted by $\overleftrightarrow{s_j}$) within the window j:

$$\overleftrightarrow{s_j} = \sum_{i=1}^{n_j} 2^{1-i}(t_{SLO} - t_i)$$

where n_j refers to the number of requests in window j, t_{SLO} is the latency target defined in SLO, and t_i is the actual response latency for request i. Note that i is the index of the requests sorted in reverse chronological order.

Since the inference time is positively correlated with the *batchsize*, we set the initial *batchsize* of a model to b^{max}/n, where n is the number of co-locating models on the current GPU, and b^{max} is the maximum *batchsize* that satisfies the SLO for that model on the current GPU. Whenever a time window j ends, *μShare* updates $\overleftrightarrow{s_j}$ and adjusts the *batchsize* accordingly. To ensure that the SLO is not violated as much as possible, the principle of adjusting the *batchsize* is to be conservative when increasing it, but aggressive when decreasing it (Figure 10).

When the SLO slack $\overleftrightarrow{s_j}$ is positive, increase the *batchsize* used in the next window $j+1$ (denoted by b_{j+1}) linearly. That is,

$$b_{j+1} = b_j + k \times \overleftrightarrow{s_j}$$

where k is a positive coefficient.

When the SLO slack $\overleftrightarrow{s_j}$ is negative, decrease the *batchsize* used in the next window $j+1$ exponentially. That is,

$$b_{j+1} = max\{b_j - e^{\lambda \times \overleftrightarrow{s_j}}, 1\}$$

where λ is a negative coefficient.

IV. Implementation

A. Overall Implementation

μShare uses PyTorch [40] as the GPU inference framework. The components of *μShare* are compiled as shared libraries (i.e., .so files) and loaded into the PyTorch process using the `LD_PRELOAD` environment variable to interact with it. To support custom kernel blocksize, we set PyTorch's blocksize limit (i.e., defined in the `C10_LAUNCH_BOUNDS(blocksize)` macro) to be consistent with the CUDA limit, i.e., 1024.

In particular, the *kernel interceptor* uses the `dlopen()` function from the `libdl` [13] library to open CUDA's dynamic link libraries (e.g., CUDA's `libcublas.so`) at runtime and return handles to the CUDA's kernel launch functions. It then uses the `dlsym()` function from the `libdl` library to obtain the addresses of the library functions and their input parameters based on the handles. The *block shaper* uses the `shm_open()` function from the `libc` [16] library to create a shared memory area. The *block shaper* exposes an interface to the *kernel interceptor*, with `kernel_process()` used for uploading parameters such as kernel blocksize. This interface uses the `mmap()` function from the `libc` library to map and modify values in shared memory. After modifying the uploaded kernel parameters, the *block shaper* returns the

parameters to the address of the shared library function obtained by the `dlsym()` function, restoring kernel execution.

B. Support for Different NVIDIA GPUs

µShare provides support for different NVIDIA GPUs. Since the number of threads per SM affects the blocksize configuration strategy, *µShare* categorizes recent GPUs into two types: **(1) GPUs with 1,536 threads per SM:** Such as NVIDIA A40, RTX 4090, and RTX 3080 Ti. Under the CUDA constraint, where the maximum blocksize is 1024, a half-plus block can be deployed on these GPUs with 1,536 threads per SM. Moreover, setting the blocksize as a multiple of 32 (i.e., the number of threads in a warp) can avoid thread resource fragmentation. Therefore, the range of the half-plus blocksize b is given by: $\{b \mid 1536/2 < b \leq 1024, b \equiv 0 \pmod{32}\}$. For example, the minimum blocksize is 800.

To determine a specific blocksize b, the *block shaper* reads the kernel's launch slack value from shared memory using the `mmap()` function. If the slack value is negative, b is set to the previous kernel's blocksize incremented by 32; otherwise, it defaults to 800. The *block shaper* then injects b into the kernel's launch address intercepted using the `dlsym()` function, thereby resuming kernel execution.

(2) GPUs with 2,048 threads per SM: Such as NVIDIA A100, A800, and H200. The half-plus strategy is not suitable for these GPUs. Even with a maximum blocksize of 1024, the CUDA scheduler still performs stacked co-location of two large blocks within the SM cores.

In this case, the optimal configuration is to set the blocksize to 1/3-plus of 2,048. This allows the CUDA scheduler to deploy two large 1/3-plus blocks from the same kernel within one SM. Subsequently, since the remaining threads in the SM are less than 1/3 of 2,048, it is not possible to deploy another large block. These remaining threads can be allocated to small blocks from another kernel with complementary hardware resource demands, thereby achieving scattered co-location.

Thus, the range of the 1/3-plus blocksize b is given by: $\{b \mid 2048/3 < b \leq 1024, b \equiv 0 \pmod{32}\}$. For example, the minimum blocksize is 704. After determining the range of blocksize, the subsequent operations are the same as in the previous case.

V. Evaluation

A. Methodology

Testbed: We deploy *µShare* across eight servers, each equipped with an Intel Xeon Gold 6338 CPU and either an NVIDIA A40 or A800 GPU, due to the different SM thread limits of these GPUs. Each CPU has 128 logical cores, with a base frequency of 2.00 GHz and a maximum frequency of 3.20 GHz, along with 251GB of server memory. We utilize PyTorch 2.2.0 as the inference framework.

(1) NVIDIA A40 GPU: The NVIDIA A40 GPU has 84 SMs and 44.784GB of memory, each SM has 1536 threads, 102,400 bytes of shared memory, and 65,536 registers. The CUDA version is 11.8.

TABLE III: The benchmark models.

Model Name	max_batch	Architecture	Weight(MB)	App Field
Llama2-7b	14	LLM	14,336	Txt Proc
GPT-2	50	LLM	548	Txt Proc
Bert	46	Transformer	507.44	Txt Proc
ResNet50-v1.5	295	CNN	97.71	Img Class
MobileNet_v2	427	CNN	13.54	Obj Det
Swin. Transf.	8	Transformer	331.49	Img Class
Vis. Transf.	77	Transformer	327.37	Img Class
Yolostiny	295	CNN	24.82	Obj Det
Resnet101	199	CNN	170.58	Obj Det
EfficientNet_B7	93	CNN	254.68	Img Class

(2) NVIDIA A800 GPU: The NVIDIA A800 GPU has 108 SMs and 80GB of memory, each SM has 2048 threads, 167,936 bytes of shared memory, and 65,536 registers. The CUDA version is 12.1.

Workloads: We select 10 commonly used models from MLPerf [42] and Pytorch benchmark [39] as shown in Table III. The selected models cover CNN, Transformer, RNN, as well as large language models (LLMs). Following the common setting in production environments [55], the SLO for inference models is set to 200ms. For Llama2-7b, which has a relatively long execution time, the request SLO is set to 400ms, and both the input and output lengths of the LLM are fixed to ten tokens.

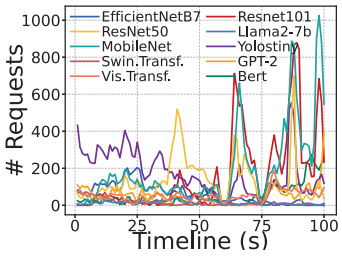

Fig. 11: The ten production trace examples.

We use Azure's inference trace (Fig. 11) from INFless [55] and scale it according to the number of GPUs used at runtime. During inference, each model runs 4 replicas, for a total of 40 replicas distributed across eight GPUs.

Comparison Systems: We compare *µShare* against the state-of-the-art systems including *INFless* and *Orion*:

Orion [46]: *Orion* co-locates kernels with different computational and memory resource demands on the GPU by controlling their launch time.

INFless [55]: *INFless* profiles the resource capacity requirements of models, and co-locates models within SMs and memory that can be accommodated by the GPU capacity.

Parameter Configuration: *µShare* sets three parameters k, λ, and β, where k is the linear trend increase parameter of the batch size, λ is the exponential trend decrease parameter of the batch size, and β is the interval parameter for delaying kernel launches. These parameters can be customized before the system runs. Through multiple experiments, we determine that the optimal values for ensuring SLO guarantees are $k = 0.05$, $\lambda = -0.1$, and $\beta = 10$.

B. Throughput Evaluation

High Throughput: *µShare increases the system throughput by 26.90%-54.09%.* We compare the throughput of *µShare*, *INFless*, and *Orion* in co-located scenarios, including the system throughput comparison (Figure 12(a)), and the normalized system throughput comparison (Figure 12(b)). The normalized

throughput is the throughput of each model divided by the unit batch size. The unit batch size is the batch size under the model's use of MPS (Multi-Process Service) [32] and the memory control interface of PyTorch to evenly distribute the unit's SM and memory resources of the GPU. *µShare* has the highest throughput in all scenarios.

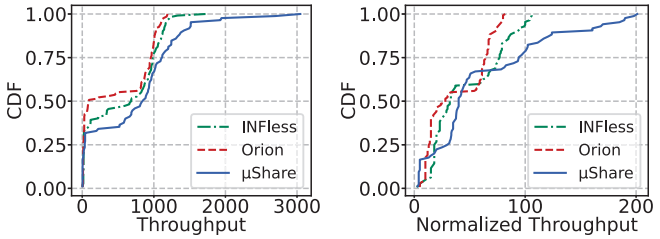

Fig. 12: (a) Actual throughput comparison. (b) Normalized throughput comparison.

Compared to *INFless*, *µShare* improves the throughput of each model by 15.02%-66.59%. The throughput of the two systems is 716.53 and 564.28, with peak throughput values of 3046 and 1722, respectively, and the normalized throughput of the two systems is 58.91 and 46.42, respectively. *µShare* shows a 26.90% improvement over *INFless*. The reason for the higher throughput of *µShare* is that it can achieve scattered co-location of kernels with different low-level hardware requirements within an SM, reducing idle hardware within the SM and thus improving resource efficiency. In contrast, *INFless* adopts stacked co-location and cannot effectively utilize the various hardware resources within the SM.

Compared to *Orion*, *µShare* improves the throughput of each model by 21.65%-92.66%. The system throughput of *Orion* is 464, with peak throughput value of 1192, and the normalized throughput is 38.23. *µShare* shows a 54.09% improvement over *Orion*. Although *Orion* is a kernel-level inference system, it adopts stacked co-location and cannot fully utilize the low-level hardware resources within the SM. Moreover, *Orion* uses a more conservative co-location strategy, allowing at most one compute-intensive kernel and one memory-intensive kernel to be co-located on the GPU to strictly control interference.

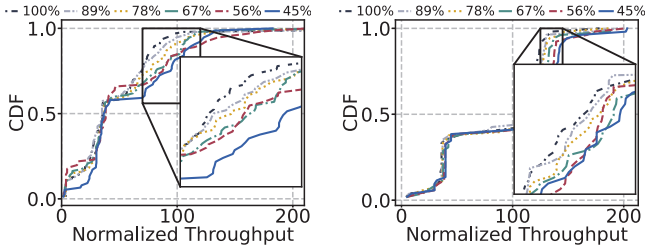

Fig. 13: The normalized throughput at different proportions of unmodifiable kernels on (a) A40 GPU and (b) A800 GPU.

Proportion of Unmodifiable Kernels: *The throughput of* µShare *increases as the proportion of unmodifiable kernels decrease.* By default, the proportion of unmodifiable kernels across 10 models is 48.37%, while modifiable kernels account for 51.63% (Section 3.3). To analyze the impact of the proportion of unmodifiable kernels on throughput, we control the number of modifiable kernels by modifying only 0, 1, 2, 3, 4, 5 out of every 5 modifiable kernels, with the remaining kernels considered unmodifiable. This gradually reduces the proportion of unmodifiable kernels from 100% to 89.67%, 79.35%, 69.02%, 58.70%, and 48.37%, and *µShare*'s throughput increases from 47.59 to 48.23, 51.42, 52.79, 54.42 and 58.81 (Figure 13(a)). Therefore, the throughput of *µShare* increases as the proportion of unmodifiable kernels decreases, and even in the worst-case scenario where all kernels are unmodifiable, *µShare*'s performance only falls back to kernel-level co-location based on resource coupling, which is equivalent to *INFless* throughput and higher than that of *Orion*.

Compare Intra-SM Co-location Technique: *Tacker* [62] is an intra-SM co-location system based on kernel fusion. It provides fused ResNet50 and BERT models. Since *Tacker* does not support SLO management across multiple inference models during co-location, for fairness we also disable *µShare*'s latency management mechanism.

Under inference trace 11, *µShare* reduces the end-to-end latency of Bert and Resnet50 by 24.71% and 16.00%, respectively (Figure 14), achieving an overall throughput improvement of 20.38%. Because kernel fusion approaches merge only adjacent intra-model kernels, whereas *µShare* co-locates kernels across tasks, providing more co-location opportunities.

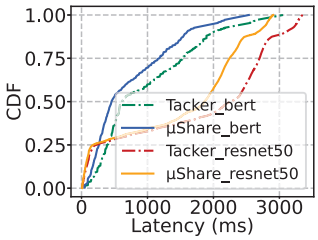

Fig. 14: Latency comparison with intra-SM co-location.

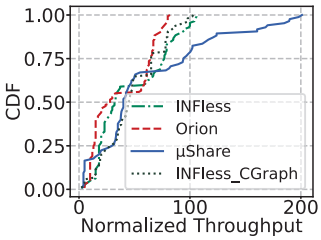

Fig. 15: Throughput compare with cuda graph technique.

Compare CUDA Graph: CUDA Graph fuses multiple kernels into a single graph for unified launch, which is beneficial when launch overhead is significant. However, in large-batch or co-location scenarios, the kernel execution time overshadows the launch time, so CUDA Graphs have little impact in our case.

We compare *µShare* with *INFless* after adding CUDA Graph optimization, as is shown in Figure 15, where *INFless* itself improves throughput by only 2.97%, while *µShare* still achieves a throughput improvement of 26.44%.

C. SLO Guarantee

Low SLO Violation: *µShare can guarantee the SLO at most time. The SLO violation rate is as low as 3.35%.* We compare the ability of three systems to guarantee the SLO of all models by repeating the experiment 20 times on NVIDIA A40 GPU for each system. The violation ratios of each model are displayed in a box plot in Figure 16. Among them, the average violation rates for the 10 models of *µShare*, *INFless*, and *Orion* are 3.35%, 2.05%, and 1.12%, respectively. Compared to existing systems, *µShare*'s SLO violation only increases by 1.30%-2.23%, but it achieves a throughput improvement of 26.90%-54.09% (Evaluation 5.2).

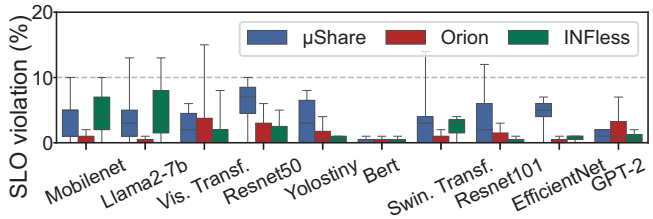

Fig. 16: SLO violation comparison of *μShare* with baselines.

μShare designs a dual-SLO guarantee mechanism for inference requests to kernels. On the inference request side, it uses SLO-based feedback control of batch size to avoid input inference requests exceeding the system load. On the kernel side, it uses feedback control based on kernel launch time to adjust the number of SM threads for each kernel, accelerating kernel execution to ensure SLO. In contrast, although *INFless* and *Orion* can adjust resource allocation based on SLO violation feedback after inference requests are completed, they cannot perform SLO detection during kernel execution, and therefore sacrifice some throughput to ensure a higher SLO satisfaction rate.

Trade-off between Throughput and SLO: *μShare* can trade off throughput and SLO violation by tuning the Batch Manager's hyperparameters k and λ. We evaluate nine configurations combining $k = \{0.05, 0.03, 0.01\}$ and $\lambda = \{-0.1, -0.15, -0.2\}$. Specifically, *μShare*_v1–v3 correspond to $\lambda = -0.1$ with $k = \{0.05, 0.03, 0.01\}$, *μShare*_v4–v6 to $\lambda = -0.15$, and *μShare*_v7–v9 to $\lambda = -0.2$.

The throughput of *μShare*_v1–v9 gradually decreases from 58.91 to 53.64 (Figure 17), while the SLO violation rate drops from 3.35% to 0.63% (Figure 18). At *μShare*_v7 (k = 0.05, λ = –0.2), *μShare* achieves an SLO violation rate of 0.84%—lower than those of the baseline systems (2.05% and 1.12%)—while maintaining a throughput improvement of 19.28%–44.83%.

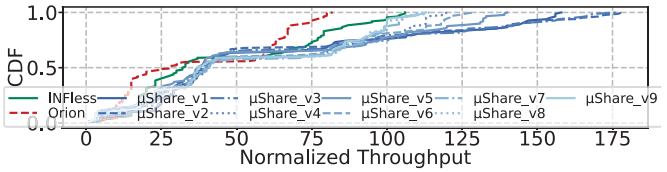

Fig. 17: Throughput under different hyperparameter settings.

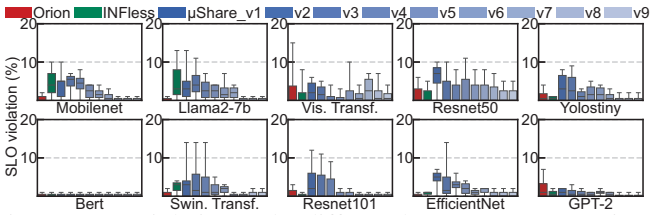

Fig. 18: SLO violation under different hyperparameter settings.

D. Latency

The end-to-end inference latency comparison between *μShare*_v7 (k = 0.05, λ = –0.2) and the two baselines is shown in Figure 19. *μShare* reduces average latency by 25.72%–29.53% and 99th-percentile latency by 25.33%–31.31%.

The lower latency of *μShare* results from intra-SM kernel parallelism, which effectively improves execution efficiency and kernel-level concurrency. In contrast, *INFless* exhibits higher execution latency because it lacks kernel-level scheduling for efficiency optimization, while *Orion*'s conservative co-location strategy limits the number of concurrent kernels, leading to increased request queuing latency.

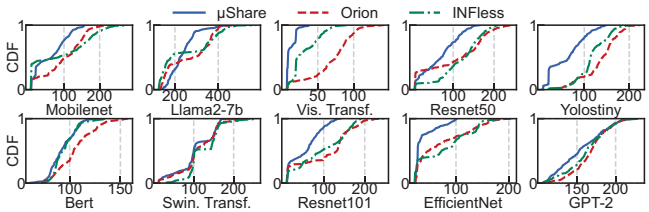

Fig. 19: Latency comparison of *μShare* and baselines.

E. Low-level Hardware Utilization

High Hardware Utilization: *μShare increases the average low-level hardward utilization by 38.53%-61.15%.* We compare the low-level hardware utilization of *μShare*, *INFless*, and *Orion* in co-located scenarios and visualize the intra-SM hardware utilization within a 200ms window of the same execution stage, as shown in Figure 20. To measure hardware utilization under co-location, we first use the NVIDIA Nsight Systems tool to record the execution time and parameters of all concurrently running kernels. We then use the NVIDIA Nsight Compute tool to individually measure the utilization of six SM hardware resources for each kernel under co-located execution. Finally, for each time interval, we aggregate the utilization of all kernels active during that interval.

The average utilization of the six low-level hardware components under *μShare*, *INFless*, and *Orion* is 15.10%, 10.90%, and 9.37%, respectively. *μShare* achieves a 38.53%–61.15% improvement over two baselines. This improvement arises because scattered co-locating enables different blocks to execute concurrently within the same SM.

F. Performance on A800 GPU

High Throughput: *μShare improves system throughput by 16.45%-52.29% on NVIDIA A800 GPUs while guaranteeing the SLO.* We compare the throughput of *μShare*, *INFless*, and *Orion* on A800 GPU. The normalized throughputs of *μShare* and *INFless* and *Orion* are 99.39 and 85.35 and 65.26 (Figure 21(a)), respectively. *μShare* improves system normalized throughput by 16.45%-52.29%. Additionally, as the proportion of unmodifiable kernels decreases from 100% to the default 48.37%, the throughput of *μShare* improves from 86.13 to 98.50 (Figure 13(b)). And the average violation rates across *μShare*, *INFless*, and *Orion* are 3.40%, 2.29%, and 1.42% (Figure 21(b)), respectively. *μShare* shows only a 1.11%-1.98% increase in SLO violations but achieves a throughput improvement of 16.45%-52.29%.

In addition, the improvement in *μShare* on the A800 GPU is slightly smaller than on the A40 GPU. This is because the A800 uses 1/3-plus shaping compared to the half-plus

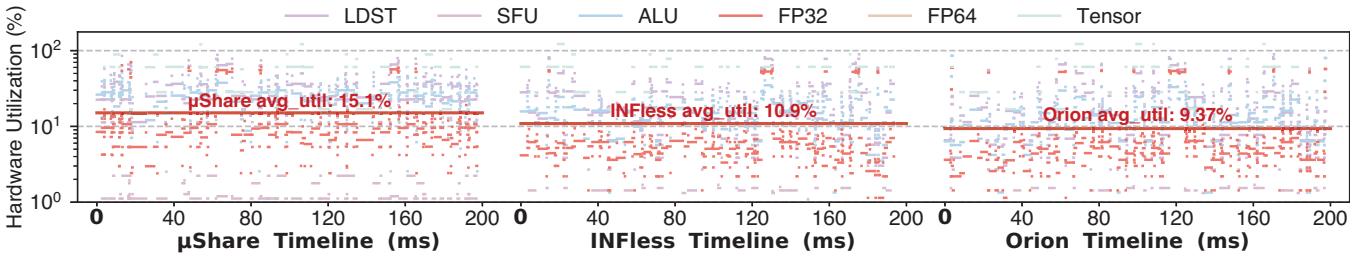

Fig. 20: Comparison of six low-level hardware utilization timelines between *μShare* and baselines.

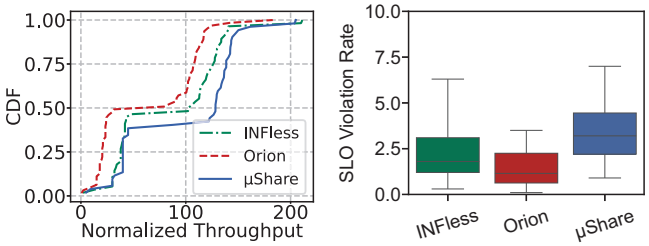

Fig. 21: (a) Normalized throughput on NVIDIA A800. (b)SLO violation on NVIDIA A800.

shaping of the A40, which increases the upper limit of blocks utilizing the same resources within a single SM core from 1/2 to 2/3 (i.e., two 1/3-plus blocks per SM), which may lead to slightly unbalanced SM thread allocation, resulting in lower improvement on the A800 compared to the A40.

G. Performance Analysis Breakdown

Shaping Improvement: *Half-plus blocksize Shaping can effectively improve the throughput by 19.40% while reducing SLO violation rate.* We analyze the impact of the *μShare* system's multiple components on throughput and SLO. We set up four breakdown scenarios: (1) *μShare*, where blocksize dynamically adjusts according to kernel launch time. (2) *μShare_shape_1024*, where blocksize is set to a fixed value, e.g., 1024. (3) *μShare w/o shape*, where blocksize is no longer adjusted. (4) *μShare w/o batch*, where inference task's batch size is no longer adjusted based on latency feedback.

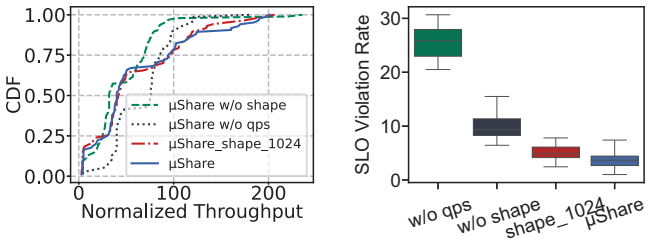

Fig. 22: Breakdown analysis of the *μShare*: impact on (a) throughput and (b) SLO violation rate.

When *μShare* fixes the blocksize of all modifiable kernels at 1024, the system throughput decreases by 3.36% in Figure 22(a) and the SLO violation rate increases by 1.32% in Figure 22(b). This is because fixing the blocksize prevents further optimization of SLO through adjusting the number of threads for the kernels. In contrast, when *μShare* does not modify the blocksize of any kernels, the system throughput decreases by 30.95% in Figure 22(a) and the SLO violation rate increases by 6.33% in Figure 22(b). This is because the system shifting from kernel-level scattered co-location to kernel-level stacked co-location, leading to inefficient utilization of intra-SM hardware resources. Furthermore, when *μShare* no longer adjusts the batch size of inference requests, the system throughput increases by 10.67% while the SLO violation rate increases by 21.90%. This is because the system loses the ability to adapt batch sizes based on inference latency feedback, resulting in a sharp throughput increase when the input load exceeds system capacity, but at the cost of significantly higher SLO violations.

H. Co-locating Scientific Computing Workloads

Broad Applicability: *μShare supports all applications that execute workloads as CUDA kernels.* With the rise of AI for science (co-locating scientific computing, inference, and training workloads) [26], [28], LLM+RAG (co-locating LLM inference and RAG) [23], [41], and other technologies [17], [58], the variety of applications running in datacenters has become increasingly diverse. As a result, we evaluate *μShare* by co-locating five scientific computing applications from the Parboil benchmark [47] with five inference models listed in Table III. Unlike inference models deployed as services, scientific computing applications are compiled into binary executables. We randomly select and execute these executables, allowing the same application to be invoked multiple times. As shown in Figure 23, *μShare* improves the overall system throughput by 18.18%–28.62%.

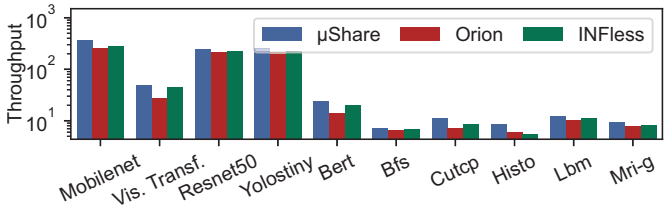

Fig. 23: Throughput evaluation under co-location scientific computing and inference applications.

These improvements are achieved because scientific computing applications typically utilize FP64 cores, whereas inference applications mainly rely on FP32 cores, LDST units, and Tensor Cores. By co-locating kernels with complementary hardware demands within the same SM, *μShare* improves hardware utilization and enhances overall system throughput.

I. Kernels and Utilization in SM at Runtime

The co-located kernels within each SM and the dominant utilization of two kernels in *μShare* (two solid lines) and one kernel in INFless (one dashed line) are shown in Fig. 24. *μShare* significantly improves hardware utilization.

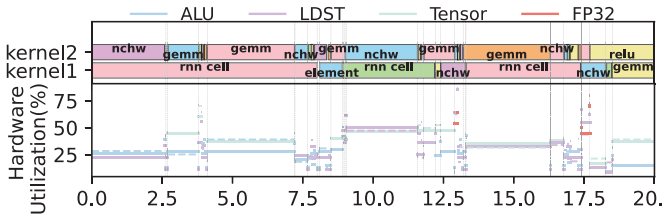

Fig. 24: Kernels and utilization in SM at runtime.

J. Overhead

Low Overhead: We measure the online execution overhead of *μShare*. The resource consumption of the system is shown in Figure 26 revealing that the CPU overhead during runtime is a mere 6.85% of a single core, suggesting an exceedingly low resource overhead. Additionally, we employ shared memory to enhance communication between the inference and control processes, the average system control overhead for a single kernel is merely 60.35 nanoseconds, as illustrated in Figure 25. Moreover, the offline profiling cost for each model ranges from 105 to 393 seconds. For Llama2-7b, which contains significantly more kernels, the profiling time is 7,160 seconds.

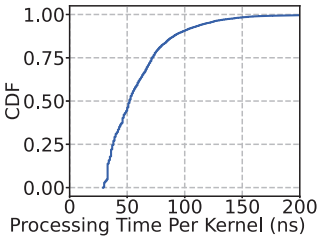

Fig. 25: The average processing time for each kernel.

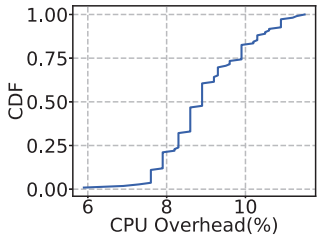

Fig. 26: The overhead during system runtime.

VI. Related Work

GPU Spatial Sharing: Spatial sharing enhances throughput by enabling multitasking on shared GPUs: NVIDIA's Multi-Instance GPU (MIG) [30], Multi-Process Service (MPS) [32] and AMD CU_masking [1] divide a single GPU into multiple partitions to serve several tasks. Orion [46] reduces interference by coupling the execution of SM-intensive and memory-intensive kernels. INFless [55] minimizes resource fragmentation through uneven allocation of SM and memory. INFless and Batchmaker [15] dynamically adjust batch sizes to guarantee latency. REEF [21] and LAX [56] ensure performance through dynamic resource allocation at runtime. Baymax [6], Prophet [5], and Gpulet [10] perform scheduling optimization by predicting QoS or interference. GPU spatial sharing techniques can improve the utilization of SMs and memory. However, they still face the problem of stacked co-location, which cannot effectively increase the utilization of hardware resources within the SM.

GPU Temporal Sharing: Temporal sharing enhance throughput by switching tasks across GPU time slices: AntMan [54] improves utilization through dynamic management of memory and compute units. IADeep [9] co-optimizes task assignments and interference mitigation. Clockwork [19] avoids long-tail delays by calculating task execution time. PipeSwitch [2] achieves efficient pipeline execution through concurrent loading and inference of model layers. However, GPU temporal sharing can only execute one kernel per unit of time, and a single kernel typically cannot fully utilize both SM and memory resources, resulting in underutilization of the GPU.

Intra-SM Sharing: Intra-SM sharing techniques enable multiple kernels to share the same SM core, fall into two primary categories: intrusive kernel modifications and intrusive hardware modifications. Intrusive kernel modifications include kernel fusion and persistent kernel. Kernel fusion, such as Tacker [62], T3 [37], Rammer [29], COMBO [4], and SpDNNs [14], merges the code of multiple kernels with different hardware resource demands into a single new kernel. Persistent kernel, such as ISPA [61], Plasticine [63], and Elastic kernel [36], launches an empty non-terminating kernel, and user kernels are then submitted to this non-terminating kernel for execution. Intrusive hardware modifications, such as CCWS [43], Prema [11], and PriorityRR [38], redesign the GPU to expose kernel scheduling interfaces and are validated through simulators. However, due to the closed-source nature of Nvidia GPU and the prohibition of accessing and modifying user code on public cloud platforms, the application scenarios of intrusive modifications are severely limited.

Non-SM Resource Sharing: Resources shared across SMs include memory bandwidth and the L2 cache. SGDRC [60] reverse-engineers GPU drivers to identify VRAM channel mappings and places data from different kernels into specific channels, achieving memory bandwidth isolation. This work is orthogonal to our intra-SM co-location approach.

VII. Conclusions

In this paper, we analyze the semantic gap between the resource demands of kernels and the resource allocation performed by the NVIDIA GPU hardware scheduler. This gap leads to stacked co-location of kernels and results in low utilization of low-level hardware resources. Without intrusive modifications to GPU, we propose a hardware–software co-design approach, *half-plus blocksize shaping*, which achieves scattered co-location of kernels. Building on this concept, we construct the *μShare* system, which effectively improves GPU resource efficiency across various NVIDIA GPUs and diverse co-location scenarios.

VIII. Acknowledgments

This work is supported by the National Key Research and Development Program of China under Grant No.2024YFB4505204, the National Natural Science Foundation of China under Grant 62372322, 62432015, and Tianjin Science and Technology Plan Project 24ZXKJGX00060.

REFERENCES

[1] "Setting the number of compute units," https://rocm.docs.amd.com/en/latest/how-to/setting-cus.html/, 2020.

[2] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin, "PipeSwitch: Fast pipelined context switching for deep learning applications," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 499–514. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/bai

[3] J. Bakita and J. H. Anderson, "Demystifying nvidia gpu internals to enable reliable gpu management," in *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2024, pp. 294–305.

[4] B. Chen, H. Zhao, W. Cui, Y. He, S. Zhang, Q. Chen, Z. Li, and M. Guo, "Maximizing the utilization of gpus used by cloud gaming through adaptive co-location with combo," in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, ser. SoCC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 265–280. [Online]. Available: https://doi.org/10.1145/3620678.3624660

[5] Q. Chen, H. Yang, M. Guo, R. S. Kannan, J. Mars, and L. Tang, "Prophet: Precise qos prediction on non-preemptive accelerators to improve utilization in warehouse-scale computers," *SIGARCH Comput. Archit. News*, vol. 45, no. 1, p. 17–32, Apr. 2017. [Online]. Available: https://doi.org/10.1145/3093337.3037700

[6] Q. Chen, H. Yang, J. Mars, and L. Tang, "Baymax: Qos awareness and increased utilization for non-preemptive accelerators in warehouse scale computers," in *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 681–696. [Online]. Available: https://doi.org/10.1145/2872362.2872368

[7] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Tvm: an automated end-to-end optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'18. USA: USENIX Association, 2018, p. 579–594.

[8] W. Chen, C. Lu, H. Xu, K. Ye, and C. Xu, "Multiplexing dynamic deep learning workloads with slo-awareness in gpu clusters," in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 589–604. [Online]. Available: https://doi.org/10.1145/3689031.3696074

[9] W. Chen, Z. Mo, H. Xu, K. Ye, and C. Xu, "Interference-aware multiplexing for deep learning in gpu clusters: A middleware approach," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3581784.3607060

[10] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving heterogeneous machine learning models on Multi-GPU servers with Spatio-Temporal sharing," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 199–216. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/choi-seungbeom

[11] Y. Choi and M. Rhu, "Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 220–233.

[12] G. Cormode, V. Shkapenyuk, D. Srivastava, and B. Xu, "Forward decay: A practical time decay model for streaming systems," in *2009 IEEE 25th International Conference on Data Engineering*, 2009, pp. 138–149.

[13] "Linux and UNIX Man Pages," https://www.unix.com/man-page/bsd/3/dlopen/, 1993.

[14] M. Dun, X. Zhang, H. Cao, Y. Zhang, J. Huang, and X. Ye, "Adaptive sparse deep neural network inference on resource-constrained cost-efficient gpus," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*, 2023, pp. 1–7.

[15] P. Gao, L. Yu, Y. Wu, and J. Li, "Low latency rnn inference with cellular batching," in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: https://doi.org/10.1145/3190508.3190541

[16] "The GNU C Library (glibc)," https://sourceware.org/glibc//, 2024.

[17] J. Gu, Y. Zhu, P. Wang, M. Chadha, and M. Gerndt, "Fast-gshare: Enabling efficient spatio-temporal gpu sharing in serverless computing for deep learning inference," in *Proceedings of the 52nd International Conference on Parallel Processing*, ser. ICPP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 635–644. [Online]. Available: https://doi.org/10.1145/3605573.3605638

[18] J. Gu, B. Zhu, M. Li, W. Li, Y. Xia, and H. Chen, "A Hardware-Software co-design for efficient Intra-Enclave isolation," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3129–3145. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/gu-jinyu

[19] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving dnns like clockwork: performance predictability from the bottom up," in *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'20. USA: USENIX Association, 2020.

[20] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 692–705.

[21] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 539–558. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/han

[22] L. Hu, X. Che, and Z. Xie, "Gpgpu cloud: A paradigm for general purpose computing," *Tsinghua Science and Technology*, vol. 18, no. 1, pp. 22–33, 2013. [Online]. Available: https://www.sciopen.com/article/10.1109/TST.2013.6449404

[23] M. A. Ibrahim, O. Kayiran, Y. Eckert, G. H. Loh, and A. Jog, "Analyzing and leveraging decoupled l1 caches in gpus," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 467–478.

[24] "Inline PTX Assembly in CUDA," https://docs.nvidia.com/cuda/inline-ptx-assembly/index.html, 2012.

[25] "intel rdt: Intel Cache Allocation Technology," https://lwn.net/Articles/634676/, 2024.

[26] J. Jiang, H. Zhang, D. Liu, J. Du, X. Yao, J. Wei, P. Chen, D. Huang, and Y. Lu, "Efficient coupling streaming ai and ensemble simulations on hpc clusters," in *Euro-Par 2024: Parallel Processing*, J. Carretero, S. Shende, J. Garcia-Blas, I. Brandic, K. Olcoz, and M. Schreiber, Eds. Cham: Springer Nature Switzerland, 2024, pp. 313–328.

[27] "Set Environment Variables," https://tldp.org/HOWTO/Oracle-9i-Fedora-3-Install-HOWTO/sect_04.html, 2005.

[28] H. Lee, M. Turilli, S. Jha, D. Bhowmik, H. Ma, and A. Ramanathan, "Deepdrivemd: Deep-learning driven adaptive molecular simulations for protein folding," in *2019 IEEE/ACM Third Workshop on Deep Learning on Supercomputers (DLS)*, 2019, pp. 12–19.

[29] L. Ma, Z. Xie, Z. Yang, J. Xue, Y. Miao, W. Cui, W. Hu, F. Yang, L. Zhang, and L. Zhou, "Rammer: Enabling holistic deep learning compiler optimizations with rTasks," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 881–897. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/ma

[30] "NVIDIA Multi-Instance GPU," https://docs.nvidia.com/datacenter/tesla/mig-user-guide//, 2020.

[31] J. Mo, J. Gopinath, and B. Reagen, "Haac: A hardware-software co-design to accelerate garbled circuits," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3579371.3589045

[32] "NVIDIA Multi-Process Service," https://docs.nvidia.com/deploy/mps//, 2013.

[33] "NVIDIA Nsight Compute," https://developer.nvidia.com/nsight-compute/, 2024.

[34] "System Management Interface SMI," https://developer.nvidia.com/system-management-interface/, 2024.

[35] "NVIDIA Nsight Systems," https://developer.nvidia.com/nsight-systems/, 2024.

[36] S. Pai, M. J. Thazhuthaveetil, and R. Govindarajan, "Improving gpgpu concurrency with elastic kernels," in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming*

Languages and Operating Systems, ser. ASPLOS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 407–418. [Online]. Available: https://doi.org/10.1145/2451116.2451160

[37] S. Pati, S. Aga, M. Islam, N. Jayasena, and M. D. Sinclair, "T3: Transparent tracking & triggering for fine-grained overlap of compute & collectives," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1146–1164. [Online]. Available: https://doi.org/10.1145/3620665.3640410

[38] S. Puthoor, X. Tang, J. Gross, and B. M. Beckmann, "Oversubscribed command queues in gpus," in *Proceedings of the 11th Workshop on General Purpose GPUs*, ser. GPGPU-11. New York, NY, USA: Association for Computing Machinery, 2018, p. 50–60. [Online]. Available: https://doi.org/10.1145/3180270.3180271

[39] "Pytorch Models And Pre-trained Weights," https://pytorch.org/vision/stable/models.html#general-information-on-pre-trained-weights/, 2017.

[40] "Pytorch," https://pytorch.org/, 2024.

[41] D. Quinn, M. Nouri, N. Patel, J. Salihu, A. Salemi, S. Lee, H. Zamani, and M. Alian, "Accelerating retrieval-augmented generation," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 15–32. [Online]. Available: https://doi.org/10.1145/3669940.3707264

[42] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "Mlperf inference benchmark," 2020.

[43] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*, 2012, pp. 72–83.

[44] R. Sabbadin, H. Fargier, and J. Lang, "Towards qualitative approaches to multi-stage decision making," *International Journal of Approximate Reasoning*, vol. 19, no. 3, pp. 441–471, 1998. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0888613X98100191

[45] A. Sarker, A. C. Canto, M. Mozaffari Kermani, and R. Azarderakhsh, "Error detection architectures for hardware/software co-design approaches of number-theoretic transform," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 7, pp. 2418–2422, 2023.

[46] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, fine-grained gpu sharing for ml applications," in *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1075–1092. [Online]. Available: https://doi.org/10.1145/3627703.3629578

[47] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," *Center for Reliable and High-Performance Computing*, vol. 127, no. 7.2, 2012.

[48] Q. Sun, Y. Liu, H. Yang, Z. Luan, and D. Qian, "Smqos: Improving utilization and energy efficiency with qos awareness on gpus," in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*, 2019, pp. 1–5.

[49] "Tensorflow Serving Models," https://www.tensorflow.org/tfx/guide/serving, 2024.

[50] "NVIDIA TensorRT," https://docs.nvidia.com/tensorrt/index.html, 2024.

[51] R. Veldema and M. Philippsen, "Parallel memory defragmentation on a gpu," in *Proceedings of the 2012 ACM SIGPLAN Workshop on Memory Systems Performance and Correctness*, ser. MSPC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 38–47. [Online]. Available: https://doi.org/10.1145/2247684.2247693

[52] R. Veldema and M. Philippsen, "Parallel memory defragmentation on a gpu," in *Proceedings of the 2012 ACM SIGPLAN Workshop on Memory Systems Performance and Correctness*, ser. MSPC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 38–47. [Online]. Available: https://doi.org/10.1145/2247684.2247693

[53] Z. Wang, J. Yang, R. Melhem, B. Childers, Y. Zhang, and M. Guo, "Simultaneous multikernel gpu: Multi-tasking throughput processors via fine-grained sharing," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 358–369.

[54] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, "AntMan: Dynamic scaling on GPU clusters for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 533–548. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/xiao

[55] Y. Yang, L. Zhao, Y. Li, H. Zhang, J. Li, M. Zhao, X. Chen, and K. Li, "Infless: a native serverless system for low-latency, high-throughput inference," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 768–781. [Online]. Available: https://doi.org/10.1145/3503222.3507709

[56] T. T. Yeh, M. D. Sinclair, B. M. Beckmann, and T. G. Rogers, "Deadline-aware offloading for high-throughput accelerators," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 479–492.

[57] C. Yu, Y. Bai, H. Yang, K. Cheng, Y. Gu, Z. Luan, and D. Qian, "Smguard: A flexible and fine-grained resource management framework for gpus," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2849–2862, 2018.

[58] W. Zhang, B. Chen, Z. Han, Q. Chen, P. Cheng, F. Yang, R. Shu, Y. Yang, and M. Guo, "PilotFish: Harvesting free cycles of cloud gaming with deep learning training," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 217–232. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/zhang-wei

[59] W. Zhang, Q. Chen, N. Zheng, W. Cui, K. Fu, and M. Guo, "Toward qos-awareness and improved utilization of spatial multitasking gpus," *IEEE Transactions on Computers*, vol. 71, no. 4, pp. 866–879, 2022.

[60] Y. Zhang, H. Yu, C. Han, C. Wang, B. Lu, Y. Li, Z. Jiang, Y. Li, X. Chu, and H. Li, "Sgdrc: Software-defined dynamic resource control for concurrent dnn inference on nvidia gpus," in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 267–281. [Online]. Available: https://doi.org/10.1145/3710848.3710863

[61] H. Zhao, W. Cui, Q. Chen, and M. Guo, "Ispa: Exploiting intra-sm parallelism in gpus via fine-grained resource management," *IEEE Transactions on Computers*, vol. 72, no. 5, pp. 1473–1487, 2023.

[62] H. Zhao, W. Cui, Q. Chen, Y. Zhang, Y. Lu, C. Li, J. Leng, and M. Guo, "Tacker: Tensor-cuda core kernel fusion for improving the gpu utilization while ensuring qos," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 800–813.

[63] H. Zhao, W. Cui, Q. Chen, J. Zhao, J. Leng, and M. Guo, "Exploiting intra-sm parallelism in gpus via persistent and elastic blocks," in *2021 IEEE 39th International Conference on Computer Design (ICCD)*, 2021, pp. 290–298.